Advanced Topics in Databases: Final Report

Amos Uwamusi up202502075 Kamil Tasarz up202512958
Sérgio Cardoso up202107918

June 4, 2026

0.1 Introduction

Everyday, we are bombarded with a sheer amount of data that has to be stored in databases and efficiently managed to extract meaningful insights. The field of database management has evolved significantly over the years, with various models and techniques being developed to handle the increasing complexity and volume of data. One of the key challenges in database management is ensuring that data is stored in a way that allows for efficient querying and retrieval. This is where the concept of database normalization comes into play.

0.2 Development

So, for this project, we used the 2021 and 2017 Portuguese local elections as the basis. The data was sourced from the official election results provided by the Portuguese government, which included detailed information on votes, candidacies, parties, and municipalities. We designed a database schema to efficiently store this data and implemented an ETL pipeline to extract, transform, and load the data into our database.

0.2.1 ETL Pipeline

0.2.2 Functions, Views, and Triggers

All functions, views, and triggers were implemented in PostgreSQL using PL/pgSQL.

- **Functions:**
  - `calculate_vote_percentage(p_candidacy_id)`: Calculate vote percentage for a candidacy.
  - `get_party_performance_in_municipality(p_election_id, p_municipality_id, p_party_acronym)`: Get the performance of a party in a specific municipality.
  - `get_top_parties(p_election_id, p_municipality_id, p_organ_id, p_limit)`: Get the top parties in a specific municipality.
  - `allocate_seats_dhondt(p_election_id, p_organ_id, p_municipality_id, p_total_seats)`: Allocate seats based on the D'Hondt method for a specific election and municipality.
  - `calculate_turnout_percentage()`: Calculate voter turnout percentage.
  - `get_or_create_party(p_party_acronym, p_party_name)`: Get or create a party based on its acronym and name.
- **Views:**
  - `vw_complete_results`: A view that provides complete election results, including vote counts, percentages, and seat allocations for each candidacy.
  - `vw_turnout_analysis`: A view that provides turnout analysis for each election and municipality.
  - `vw_candidacy_details`: A view that provides complete candidacy information, including vote percentages and seat allocations.
  - `vw_municipality_summary`: A view that summarizes election results by municipality, including total votes, turnout, and seat allocations.
  - `vw_stg_missing_data`: A view that identifies any missing data in the staging tables, such as missing votes or candidacy information.
  - `vw_stg_potential_duplicates`: A view that identifies potential duplicate records in the staging tables, such as duplicate votes or candidacies.
- **Triggers:**
  - `trg_turnout_percentages`: Automatically create turnout percentages on INSERT or UPDATE of votes.
  - `trg_vote_percentages`: Automatically vote percentages on INSERT or UPDATE of votes.

- `trg_audit_log`: Audit log for important table changes (e.g., when a new election is added or when votes are updated).
- `trg_audit_candidacy`: Audit log for candidacy changes (e.g., when a new candidacy is added or updated).
- `trg_audit_vote_result`: Audit log for vote result changes (e.g., when votes are updated or when seat allocations are changed).

0.2.3 Analytical Queries

After implementing the functions, views, and triggers, we tested them using SQL queries to ensure they were working correctly and efficiently. We also analyzed the results to gain insights into the election data, such as identifying which parties had the most votes and how the seats were allocated based on the D'Hondt method.

0.2.4 Frontend Overview

For the frontend, we used Flask to create a web application that allows users to interact with the election data. The frontend provides a user-friendly interface to query the database and visualize the results, such as displaying vote percentages and seat allocations in a more intuitive way.

In it, we

0.2.5 Known Limitations

0.2.6 Short Statement of Individual Contributions

- **Amos Uwamusi:**
  - Developed the ETL pipeline to extract, transform, and load the election data into the database.
  - Implemented functions to calculate vote percentages and allocate seats based on the D'Hondt method.
  - Created views to provide complete election results and turnout analysis.
  - Developed triggers to automatically calculate percentages and maintain data integrity.
- **Kamil Tasarz:**
  - Designed and implemented the database schema to store the election data efficiently.
  - Created functions to analyze party performance and get top parties in specific municipalities.
  - Developed views to summarize election results by municipality and identify missing data.
  - Implemented triggers for audit logging of important table changes.
- **Sérgio Cardoso:**
  - Adapted the data handling processes to manage multiple elections and municipalities, ensuring scalability and flexibility.
  - Developed both the report and the slides for the presentation, summarizing the project and its outcomes effectively.

0.3 Conclusion

In conclusion, this project provided us with an opportunity to apply the concepts and techniques we learned in the Advanced Topics in Databases course to a real-world dataset. We designed and implemented a robust ETL pipeline to handle the election data, created functions, views, and triggers to perform necessary calculations and maintain data integrity, and developed a frontend to allow users to interact with the data in a meaningful way. Through this process, we gained valuable insights into the election data and were able to analyze it in various ways, such as identifying which parties had the most votes and how the seats were allocated based on the D'Hondt method. Overall, this project was a great learning experience that allowed us to deepen our understanding of databases and their applications in real-world scenarios.

.1 Appendix